Running the Project 2 tests locally

CROSSFADE, info579_group_1 (Herling, Nathan; Pabari, Nidhi Nilesh). For **single_table.sql**: 71 tests, last run 2026-11-08 20:01:18: 71 passed, 0 failed.

Nothing here is taken on trust: the tests compute every expected answer in plain Python from the Project 1 sample data (`data.csv`), run each query on your own MySQL server, and compare the two row for row. Commands are for Windows PowerShell; the macOS / Linux form follows where it differs.

1. What you need

Item	Version	Check it
MySQL Server (a running service, not only Workbench)	8.0.16 or later (tested 8.0.46, 8.4.9)	`Get-Service *mysql* (Status: Running)`
The MySQL root password		`Workbench: connecting to Local instance asks for it`
Python	3.10 or later (tested 3.13)	`python --version`
MySQL Workbench (optional)	8.0 or later	`only to build the database by hand (step 4)`

The `mysql` command-line client is **not** needed: Workbench or the tests themselves (`--fresh`) build the database.

2. Get the files

Repo: `project2/tests/crossfade_p2_starter_kit.zip` (the **starter kit**; the site no longer offers it for download). It holds everything except the SQL you write:

File	What it is
`single_table.sql`	the file under test: blank in the starter kit (comments only). To test the answer key, copy project2/deliverables/single_table.sql here, replacing the blank one.
`database_implementation.sql`	builds the crossfade database with the Project 1 sample data
`data.csv`	the Project 1 sample data; the tests compute the expected answers from it
`use_cases.json`	the use cases the queries answer
`conftest.py, oracle.py, test_single_table.py`	the tests (oracle.py = the expected answers, in Python)
`pytest.ini, requirements.txt`	pytest settings; the three Python packages needed
`README.md, STARTER.md`	how the tests work; the rules the comments fix
`run_tests_locally.pdf, use_case_reference.pdf`	this guide; every use case with its input and expected output

Unzip it into a folder of your own (the zip holds one folder, `crossfade_p2_starter`):

```
cd <your working folder>
Expand-Archive -Force $HOME\Downloads\crossfade_p2_starter_kit.zip .
cd .\crossfade_p2_starter
Get-ChildItem | ft Name, Length
```

Windows Explorer can open a .zip without extracting it; Python cannot run files from inside one, so extract first.

3. Set up Python (once per folder)

```
python -m venv .venv
.venv\Scripts\python -m pip install -r requirements.txt
.venv\Scripts\python -c "import pytest, pymysql, cryptography; print('ok', pytest.__version__)"
```

macOS / Linux: `python3 -m venv .venv`, then `.venv/bin/python ...`. `cryptography` is required: MySQL 8 signs in with caching_sha2_password, which PyMySQL cannot do without it. Calling `.venv\Scripts\python` directly avoids activating the venv (no PowerShell execution-policy prompt).

4. Build the crossfade database

Option A, Workbench. Connect to Local instance; File > Open SQL Script > `database_implementation.sql`; click the lightning-bolt **Run** (whole script). The Output pane ends with *22 statements executed successfully*; the ten INSERTs report 10 + 8 + 8 + 5 + 15 + 8 + 27 + 270 + 19 + 31 = 401 rows. Right-click the Schemas list > Refresh All to see `crossfade`. Check:

```
SELECT COUNT(*) FROM crossfade.genre_probability;   -- 270
SELECT COUNT(*) FROM information_schema.tables WHERE table_schema = 'crossfade';   -- 10
```

Option B, let the tests build it: add `--fresh` to the pytest command in step 5. It drops and rebuilds `crossfade` from database_implementation.sql, printing *--fresh: statement k/22* as it goes.

Rebuilding is always safe: the script drops the database first. Before testing, the tests confirm every table holds exactly the rows in data.csv and stop with a message if not.

5. Run the tests

```
$env:CROSSFADE_DB_PASSWORD = "your-root-password"
.venv\Scripts\python -m pytest            # add --fresh to rebuild the database first
```

macOS / Linux: `export CROSSFADE_DB_PASSWORD='...'` then `.venv/bin/python -m pytest`. The header lists the four input files found and the mode; then one line per test with a running percentage, then the summary:

```
crossfade data       ...\data.csv
crossfade impl       ...\database_implementation.sql
crossfade sql        ...\single_table.sql
crossfade use_cases  ...\use_cases.json
crossfade mode       test the existing `crossfade` database
collected 71 items
test_single_table.py::test_comment_contract[S1] PASSED      [  1%]
...
=================== 71 passed in 0.35s ====================
```

6. Read the results

Test	Per	What a failure means
`test_comment_contract`	query	the comment is missing a field the tests read (tag, Use cases, Business need, Business question, Requirements, Output), or Output differs from the oracle's columns
`test_read_only_select`	query	not a single SELECT, uses SELECT *, or reaches outside crossfade
`test_reads_exactly_one_table`	query	MySQL's EXPLAIN shows a table other than the one after the tag
`test_requirements_match_declared`	query	Requirements: lists (a)-(e) the SQL does not use, or misses one it does
`test_columns_match_use_cases`	query	reads a column its use cases do not declare in use_cases.json
`test_result_matches_oracle`	query	columns, row count, a value or the row order differ from the expected answer
`test_result_is_deterministic`	query	the ORDER BY includes no unique key (or GROUP BY columns), so tied rows could come back in any order; or two runs differ
`test_s#_...`	case	logic the sample data never reaches (a row added or changed, rolled back) gives the wrong rows
`test_expected_error_is_raised[E1]`	error	the duplicate upload was not refused with error 1062, or the table changed
`coverage tests (4)`	file	a table never queried, a requirement met fewer than twice, a use case unanswered, tags out of order or an [E#] before an [S#]

Each run writes `results/single_table_results.json`: every query's rows, the expected rows, every case's inputs, and every outcome.

7. When something goes wrong

Message	Cause and fix
`set CROSSFADE_DB_PASSWORD ...`	step 5's first line was not run in this PowerShell window
`cannot open the crossfade database ... (2003, Can't connect)`	MySQL is not running (Get-Service *mysql*; Start-Service from an admin window) or listens on another port: add --db-port 3307 (or yours)
`(1045, Access denied for user 'root')`	wrong password in CROSSFADE_DB_PASSWORD
`(1049, Unknown database 'crossfade')`	build it: step 4, or rerun with --fresh

Message	Cause and fix
`crossfade.<table> has N rows but data.csv has M`	the database was changed or built from another file: rebuild (--fresh)
`cannot find data.csv; looked in: ...`	a file is missing from the folder; or pass --data PATH (also --impl, --sql, --use-cases)
`RuntimeError: 'cryptography' package is required`	run the pip install line of step 3 inside the venv
`[S1] has no statement in single_table.sql yet`	starter kit: that query is not written yet (expected while you work)
`'mysql' is not recognized`	harmless: the command-line client is not needed (Workbench or --fresh)

8. Writing the queries (starter kit)

The comment above each placeholder is the specification. The tests read its **[S#] TABLE** tag, **Use cases**, **Requirements**, **Output** and (for [E1]) **Expects** lines: keep them. Your column names and their order must equal **Output**; your rows must equal oracle.py's, in order, so every ORDER BY must break its ties. Every day count is measured to the fixed report date `'2026-09-30'`. [E1] is a write MySQL must refuse; write it inside the block comment under its comment, so it stays DISABLED and the file runs clean (the tests run it on their own). The use_case_reference.pdf in this kit shows each query's input and expected output.